Phase 3 Notes: Decreasing Multiple-Call Recurrences

1. Main Goal of Phase 3

In Phase 3, we study recursive functions that call themselves **more than once**.

In Phase 1 and Phase 2, the main example had only **one recursive call**:

```
printNumber(n - 1);
```

That kind of recursion creates a simple chain:

```
n -> n - 1 -> n - 2 -> n - 3 -> ... -> 0
```

In Phase 3, the function makes **two recursive calls**:

```
algorithm2(n - 1);
algorithm2(n - 1);
```

This creates a branching structure, like a tree.

The main recurrence in this phase is:

```
T(n) = 2T(n - 1) + 1
T(0) = 1
```

The final time complexity is:

```
O(2^n)
```

The main lesson is:

> One decreasing recursive call creates a chain.
> Two decreasing recursive calls create a tree.
> A tree that doubles at each level grows exponentially.

---

2. Current Progress Before Phase 3

Before this phase, you already learned these ideas.

From Phase 1

You learned how to look at recursive code and write a recurrence relation.

Example:

```
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}
```

The recurrence is:

```
T(n) = T(n - 1) + 1
T(0) = 1
```

You learned that:

- `T(n)` means the running time for input size `n`.
- `T(n - 1)` means the time taken by the smaller recursive call.
- `+1` means constant work done in the current call.
- `T(0) = 1` is the base case.

From Phase 2

You learned how to solve a one-call decreasing recurrence using successive substitution.

Example:

```
T(n) = T(n - 1) + 1
T(n) = T(n - 2) + 2
T(n) = T(n - 3) + 3
T(n) = T(n - k) + k
```

Then you used the base case:

```
n - k = 0
k = n
```

So:

```
T(n) = T(0) + n
T(n) = 1 + n
T(n) = O(n)
```

You also learned these one-call patterns:

Recurrence	Final Time
`T(n) = T(n - 1) + 1`	`O(n)`
`T(n) = T(n - 1) + n`	`O(n^2)`
`T(n) = T(n - 1) + n^2`	`O(n^3)`

Now Phase 3 changes one important thing:

> Instead of one recursive call, the function makes two recursive calls.

---

3. What Is a Decreasing Multiple-Call Recurrence?

A recurrence is called **decreasing** when the input becomes smaller by subtraction.

Examples:

```
n becomes n - 1
n becomes n - 2
n becomes n - 3
```

A recurrence is called **multiple-call** when the function calls itself more than once.

Example:

```
algorithm2(n - 1);
algorithm2(n - 1);
```

This function calls itself two times.

So this is a:

```
Decreasing multiple-call recurrence
```

because:

1. The input decreases from `n` to `n - 1`.
2. The function makes more than one recursive call.

---

4. Main Code Example of Phase 3

```
#include <iostream>
using namespace std;

void algorithm2(int n) {
    if (n > 0) {
        cout << n << " ";
        algorithm2(n - 1);
        algorithm2(n - 1);
    }
}

int main() {
    algorithm2(3);
    return 0;
}
```

This function:

1. Checks whether `n > 0`.
2. Prints the current value of `n`.
3. Calls itself with `n - 1`.
4. Calls itself again with `n - 1`.
5. Stops when `n = 0`.

---

5. Breaking the Code Into Parts

Look at the important part:

```
if (n > 0) {
    cout << n << " ";
```

```
    algorithm2(n - 1);
    algorithm2(n - 1);
}
```

Inside each working call, three things happen.

Part 1: Base Condition

```
if (n > 0)
```

The function continues only while `n > 0`.

When `n = 0`, the condition becomes false:

```
0 > 0 is false
```

So the function stops.

The base case is:

```
T(0) = 1
```

This means the stopping call takes constant time.

Part 2: Current Work

```
cout << n << " ";
```

This prints one value.

Printing one value takes constant time.

So the current work is:

```
+1
```

Part 3: Recursive Calls

```
algorithm2(n - 1);
algorithm2(n - 1);
```

There are two recursive calls.

Each call receives input `n - 1`.

So the recursive part is:

```
T(n - 1) + T(n - 1)
```

This can be simplified as:

```
2T(n - 1)
```

---

6. Writing the Recurrence Relation

The total time is:

```
Time of first recursive call
+ Time of second recursive call
+ Current work
```

So:

```
T(n) = T(n - 1) + T(n - 1) + 1
```

Since both recursive calls are the same:

```
T(n) = 2T(n - 1) + 1
```

Base case:

```
T(0) = 1
```

Full recurrence:

```
T(n) = 2T(n - 1) + 1, for n > 0
T(0) = 1
```

---

7. Meaning of Each Part of the Recurrence

The recurrence is:

```
T(n) = 2T(n - 1) + 1
```

Part	Meaning
`T(n)`	Time taken for input size `n`
`2`	There are two recursive calls
`T(n - 1)`	Each recursive call works on input `n - 1`
`+1`	Constant work done in the current call
`T(0) = 1`	Base case takes constant time

In simple words:

> For input `n`, the function does one small job now, then creates two smaller problems of size `n - 1`.

---

8. Difference Between Phase 2 and Phase 3

Phase 2: One Recursive Call

```cpp
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}
```

Recurrence:

```
T(n) = T(n - 1) + 1
```

Call shape:

```
n -> n - 1 -> n - 2 -> n - 3 -> ... -> 0
```

This is a chain.

Final time:

```
O(n)
```

Phase 3: Two Recursive Calls

```
void algorithm2(int n) {
    if (n > 0) {
        cout << n << " ";
        algorithm2(n - 1);
        algorithm2(n - 1);
    }
}
```

Recurrence:

```
T(n) = 2T(n - 1) + 1
```

Call shape:

```
        n
      /
   n - 1  n - 1
    / \    /
  ... ... ... ...
```

This is a tree.

Final time:

```
O(2^n)
```

---

9. Why Two Calls Are Much More Expensive

A beginner might think:

> Two recursive calls means the program is only twice as slow.

But that is not correct.

The function does not just double once.

It doubles again and again at every level.

The number of calls grows like this:

```
1, 2, 4, 8, 16, 32, ...
```

This kind of growth is called:

```
Exponential growth
```

That is why the answer becomes:

```
O(2^n)
```

---

10. Dry Run of `algorithm2(3)`

Call:

```
algorithm2(3);
```

Step 1: First call

```
algorithm2(3)
```

Since `3 > 0`, it prints:

```
3
```

Then it calls:

```
algorithm2(2)
algorithm2(2)
```

Step 2: First `algorithm2(2)`

The first `algorithm2(2)` prints:

```
2
```

Then it calls:

```
algorithm2(1)
algorithm2(1)
```

Step 3: First `algorithm2(1)`

The first `algorithm2(1)` prints:

```
1
```

Then it calls:

```
algorithm2(0)
algorithm2(0)
```

Both `algorithm2(0)` calls stop because `0 > 0` is false.

Step 4: Second `algorithm2(1)` under the first `algorithm2(2)`

It prints:

```
1
```

Then it calls:

```
algorithm2(0)
algorithm2(0)
```

Both stop.

Step 5: Second `algorithm2(2)`

Now the second `algorithm2(2)` runs.

It prints:

```
2
```

Then it calls:

```
algorithm2(1)
algorithm2(1)
```

Each `algorithm2(1)` prints `1` and then stops after calling `algorithm2(0)` twice.

Final Output

```
3 2 1 1 2 1 1
```

---

11. Call Tree for `algorithm2(3)`

The call tree looks like this:

```
              3
            /
          2       2
        /  \     /
      1     1  1     1
```

Each node represents a working call.

The `0` calls are not shown in this tree because they only stop.

The number of working calls is:

```
1 + 2 + 4 = 7
```

That matches the output:

```
3 2 1 1 2 1 1
```

There are 7 printed values.

---

12. Work by Level

For `algorithm2(3)`:

Level	Calls at This Level	Work per Call	Total Work at Level
Level 0	1	1	1
Level 1	2	1	2
Level 2	4	1	4

Total work:

```
1 + 2 + 4 = 7
```

For `algorithm2(4)`:

Level	Calls at This Level
Level 0	1

Level	Calls at This Level
Level 1	2
Level 2	4
Level 3	8

Total work:

```
1 + 2 + 4 + 8 = 15
```

For `algorithm2(5)`:

```
1 + 2 + 4 + 8 + 16 = 31
```

The pattern is:

```
1 + 2 + 4 + ... + 2^n
```

This is why the time is exponential.

---

13. Successive Substitution Method

Now we solve the recurrence formally.

Start with:

```
T(n) = 2T(n - 1) + 1
```

First Substitution

We know:

```
T(n - 1) = 2T(n - 2) + 1
```

Substitute this into the original recurrence:

```
T(n) = 2[2T(n - 2) + 1] + 1
```

Multiply:

```
T(n) = 4T(n - 2) + 2 + 1
```

Write `4` as `2^2`:

```
T(n) = 2^2T(n - 2) + 2 + 1
```

Second Substitution

We know:

```
T(n - 2) = 2T(n - 3) + 1
```

Substitute:

```
T(n) = 2^2[2T(n - 3) + 1] + 2 + 1
```

Multiply:

```
T(n) = 2^3T(n - 3) + 2^2 + 2 + 1
```

Third Substitution

The next step would become:

```
T(n) = 2^4T(n - 4) + 2^3 + 2^2 + 2 + 1
```

Now we can see the pattern.

---

14. Substitution Pattern

After repeated substitution, we get:

```
T(n) = 2^kT(n - k) + 2^(k - 1) + 2^(k - 2) + ... + 2 + 1
```

This means after `k` substitution steps:

- The recursive input becomes `n - k`.
- The coefficient in front of the recursive term becomes `2^k`.
- The added work forms a geometric progression.

---

15. Using the Base Case

The base case is:

```
T(0) = 1
```

So we want:

```
T(n - k)
```

to become:

```
T(0)
```

That means:

```
n - k = 0
```

Solve for `k`:

```
k = n
```

So the recurrence reaches the base case after `n` levels.

---

16. Substitute `k = n`

The pattern is:

```
T(n) = 2^kT(n - k) + 2^(k - 1) + 2^(k - 2) + ... + 2 + 1
```

Put:

```
k = n
```

So:

```
T(n) = 2^nT(n - n) + 2^(n - 1) + 2^(n - 2) + ... + 2 + 1
```

Since:

```
n - n = 0
```

we get:

```
T(n) = 2^nT(0) + 2^(n - 1) + 2^(n - 2) + ... + 2 + 1
```

Since:

```
T(0) = 1
```

we get:

```
T(n) = 2^n + 2^(n - 1) + 2^(n - 2) + ... + 2 + 1
```

Rewrite in increasing order:

```
T(n) = 1 + 2 + 4 + ... + 2^(n - 1) + 2^n
```

---

17. Geometric Progression

The expression:

```
1 + 2 + 4 + ... + 2^n
```

is called a geometric progression, or GP series.

A geometric progression is a sequence where each term is multiplied by the same number.

Here, each term is multiplied by `2`:

```
1, 2, 4, 8, 16, ...
```

The formula is:

```
1 + 2 + 4 + ... + 2^n = 2^(n + 1) - 1
```

So:

```
T(n) = 2^(n + 1) - 1
```

In Big O notation, constants and smaller terms are ignored.

So:

```
T(n) = O(2^n)
```

---

18. Final Time Complexity

The recurrence is:

```
T(n) = 2T(n - 1) + 1
```

The final result is:

```
T(n) = 2^(n + 1) - 1
```

So the time complexity is:

```
O(2^n)
```

This is called exponential time.

---

19. Why the Time Is `O(2^n)`

The function creates a binary tree of calls.

At each level, the number of calls doubles:

```
Level 0: 1 call
Level 1: 2 calls
Level 2: 4 calls
Level 3: 8 calls
Level 4: 16 calls
```

The total work is:

```
1 + 2 + 4 + 8 + 16 + ...
```

This grows like:

```
2^n
```

So the final time is:

```
O(2^n)
```

---

20. Stack Space Complexity

The time complexity is:

```
O(2^n)
```

But the stack space is:

```
O(n)
```

This may feel confusing at first.

The total number of calls is exponential, but the computer does not keep all calls active at the same time.

The recursion goes deep along one path first:

```
algorithm2(n)
algorithm2(n - 1)
algorithm2(n - 2)
algorithm2(n - 3)
...
algorithm2(0)
```

The longest path has length about `n`.

So the maximum number of calls waiting on the stack at one time is about `n`.

Therefore:

```
Stack space = O(n)
```

---

21. Time Complexity vs Space Complexity

For Phase 3:

Recurrence	Time Complexity	Stack Space
`T(n) = 2T(n - 1) + 1`	`O(2^n)`	`O(n)`

Why time is `O(2^n)`:

- The function creates two calls at every working call.
- The number of total calls doubles at each level.
- Total calls grow exponentially.

Why space is `O(n)`:

- The deepest path from `n` to `0` has length `n`.
- The stack only stores the active path, not the whole tree at once.

---

22. Complete Coding Program for Phase 3

```cpp
#include <iostream>
using namespace std;

void algorithm2(int n) {
    if (n > 0) {
        cout << n << " ";        // constant work: +1
        algorithm2(n - 1);       // first recursive call
        algorithm2(n - 1);       // second recursive call
    }
}

int main() {
    int n = 3;

    cout << "Output: ";
    algorithm2(n);

    cout << endl;
    return 0;
}
```

Output:

```
Output: 3 2 1 1 2 1 1
```

Recurrence:

```
T(n) = 2T(n - 1) + 1
T(0) = 1
```

Time complexity:

```
O(2^n)
```

Stack space:

```
O(n)
```

---

23. Full Solution in One Place

Start:

```
T(n) = 2T(n - 1) + 1
```

Substitute once:

```
T(n) = 2[2T(n - 2) + 1] + 1
T(n) = 2^2T(n - 2) + 2 + 1
```

Substitute again:

```
T(n) = 2^3T(n - 3) + 2^2 + 2 + 1
```

Pattern:

```
T(n) = 2^kT(n - k) + 2^(k - 1) + ... + 2 + 1
```

Use base case:

```
n - k = 0
k = n
```

Substitute `k = n`:

```
T(n) = 2^nT(0) + 2^(n - 1) + ... + 2 + 1
```

Since `T(0) = 1`:

```
T(n) = 2^n + 2^(n - 1) + ... + 2 + 1
```

GP sum:

```
1 + 2 + 4 + ... + 2^n = 2^(n + 1) - 1
```

Final:

```
T(n) = 2^(n + 1) - 1
```

Big O:

```
T(n) = O(2^n)
```

---

24. Beginner-Friendly Explanation of the Solution

The recurrence:

```
T(n) = 2T(n - 1) + 1
```

means:

> Do one small job now, then solve two smaller problems.

For input `n = 3`, the working calls are:

```
1 call of size 3
2 calls of size 2
4 calls of size 1
```

Total:

```
1 + 2 + 4 = 7
```

For larger `n`, the number of calls keeps doubling:

```
1 + 2 + 4 + 8 + ... + 2^n
```

This is why the answer is:

```
O(2^n)
```

---

25. Important Terms in Phase 3

Multiple Recursive Calls

A function has multiple recursive calls when it calls itself more than once.

Example:

```
algorithm2(n - 1);
algorithm2(n - 1);
```

This creates branching recursion.

Branching Recursion

Branching recursion happens when one function call creates two or more new function calls.

Example:

```
       n
     /
  n - 1  n - 1
```

This is different from one-call recursion, which creates a chain.

Recursion Tree

A recursion tree is a tree diagram that shows how recursive calls split.

For `algorithm2(3)`:

```
            3
          /
         2     2
```

```
          / \   /
         1   1 1   1
```

Each node is a function call.

Geometric Progression

A geometric progression is a sequence where each term is multiplied by the same value.

Example:

```
1, 2, 4, 8, 16, ...
```

In Phase 3, the total work forms this GP series:

```
1 + 2 + 4 + ... + 2^n
```

Its sum is:

```
2^(n + 1) - 1
```

Exponential Time

Exponential time means the running time grows by repeated multiplication.

For this phase:

```
O(2^n)
```

This grows much faster than:

```
O(n)
O(n^2)
O(n^3)
```

Recursion Depth

Recursion depth means the longest path of recursive calls.

For `algorithm2(n)`, one path is:

```
n -> n - 1 -> n - 2 -> ... -> 0
```

So the depth is:

```
O(n)
```

Total Calls

Total calls means all calls in the entire recursion tree.

For this phase, total calls grow like:

```
1 + 2 + 4 + ... + 2^n
```

So total calls are:

```
O(2^n)
```

---

26. Common Beginner Mistakes

Mistake 1: Writing Only One Recursive Call in the Recurrence

Wrong:

```
T(n) = T(n - 1) + 1
```

Correct:

```
T(n) = 2T(n - 1) + 1
```

Why?

Because the code has two recursive calls:

```
algorithm2(n - 1);
algorithm2(n - 1);
```

Mistake 2: Thinking Two Calls Means Only Double Time

A beginner may think:

> Two recursive calls means it is just twice as much work.

But the calls double at every level:

```
1, 2, 4, 8, 16, ...
```

So the time becomes exponential:

```
O(2^n)
```

Mistake 3: Confusing Recursion Depth With Total Calls

Depth:

```
O(n)
```

Total calls:

```
O(2^n)
```

The depth is only one path down the tree.

The total calls count the entire tree.

Mistake 4: Forgetting the Current Work

Some students write:

```
T(n) = 2T(n - 1)
```

But the function also prints once:

```
cout << n << " ";
```

So we include `+1`:

```
T(n) = 2T(n - 1) + 1
```

Mistake 5: Forgetting the Base Case

The base case is when the function stops.

In code:

```
if (n > 0)
```

The function stops when:

```
n = 0
```

So:

```
T(0) = 1
```

Mistake 6: Stopping Substitution Too Early

Some students stop at:

```
T(n) = 2^kT(n - k) + 2^(k - 1) + ... + 1
```

But this is not the final answer.

You still need to use the base case:

```
n - k = 0
k = n
```

Then substitute `k = n`.

Mistake 7: Forgetting the GP Formula

You need the GP formula:

```
1 + 2 + 4 + ... + 2^n = 2^(n + 1) - 1
```

Without this, it is harder to simplify the final expression.

---

27. Step-by-Step Checklist for Phase 3 Problems

When you see a decreasing multiple-call recursive function, follow these steps.

Step 1: Find the Base Case

Example:

```
if (n > 0)
```

The function stops when:

```
n = 0
```

So:

```
T(0) = 1
```

Step 2: Count Recursive Calls

Example:

```
algorithm2(n - 1);
algorithm2(n - 1);
```

There are two recursive calls.

So the recurrence has:

```
2T(...)
```

Step 3: Track How the Input Changes

The input changes from:

```
n
```

to:

```
n - 1
```

So the recursive part is:

```
2T(n - 1)
```

Step 4: Count Extra Work

The function prints once:

```
cout << n << " ";
```

So extra work is:

```
+1
```

Step 5: Write the Recurrence

```
T(n) = 2T(n - 1) + 1
T(0) = 1
```

Step 6: Substitute Repeatedly

```
T(n) = 2T(n - 1) + 1
T(n) = 2^2T(n - 2) + 2 + 1
T(n) = 2^3T(n - 3) + 2^2 + 2 + 1
```

Step 7: Find the Pattern

```
T(n) = 2^kT(n - k) + 2^(k - 1) + ... + 2 + 1
```

Step 8: Use the Base Case

```
n - k = 0
k = n
```

Step 9: Substitute Back

```
T(n) = 2^nT(0) + 2^(n - 1) + ... + 2 + 1
```

Since `T(0) = 1`:

```
T(n) = 2^n + 2^(n - 1) + ... + 2 + 1
```

Step 10: Use the GP Sum

```
1 + 2 + 4 + ... + 2^n = 2^(n + 1) - 1
```

So:

```
T(n) = 2^(n + 1) - 1
```

Final Big O:

```
O(2^n)
```

---

28. Mini Practice Example 1

Find the recurrence and time complexity.

```
void test(int n) {
    if (n > 0) {
        cout << "*";
        test(n - 1);
        test(n - 1);
    }
}
```

Solution

Base case:

```
T(0) = 1
```

There are two recursive calls:

```
test(n - 1);
test(n - 1);
```

Each call has input `n - 1`.

The current work is printing once, so it is `+1`.

Recurrence:

```
T(n) = 2T(n - 1) + 1
T(0) = 1
```

Time complexity:

```
O(2^n)
```

Stack space:

```
O(n)
```

---

29. Mini Practice Example 2

Find the recurrence.

```
void example(int n) {
    if (n > 0) {
        cout << n;
        example(n - 1);
        example(n - 1);
        example(n - 1);
    }
}
```

Solution

There are three recursive calls.

Each call has input `n - 1`.

The current work is constant.

So:

```
T(n) = 3T(n - 1) + 1
T(0) = 1
```

This would grow like:

```
O(3^n)
```

This example helps you see the pattern:

Number of Calls	Recurrence	Final Time
1 call	`T(n) = T(n - 1) + 1`	`O(n)`
2 calls	`T(n) = 2T(n - 1) + 1`	`O(2^n)`
3 calls	`T(n) = 3T(n - 1) + 1`	`O(3^n)`

---

30. Simple Way to Remember Phase 3

For this code pattern:

```
function(n) {
    if (n > 0) {
        do one small work;
        function(n - 1);
        function(n - 1);
    }
}
```

The recurrence is:

```
T(n) = 2T(n - 1) + 1
```

The time complexity is:

```
O(2^n)
```

The stack space is:

```
O(n)
```

Memory trick:

```
One call = chain
Two calls = tree
Tree doubles = exponential time
```

---

31. Phase 3 Final Summary

In Phase 3, you learned how to analyze decreasing recursive functions that make multiple recursive calls.

The main code was:

```
void algorithm2(int n) {
    if (n > 0) {
        cout << n << " ";
        algorithm2(n - 1);
        algorithm2(n - 1);
    }
}
```

The recurrence is:

```
T(n) = 2T(n - 1) + 1
T(0) = 1
```

Using substitution:

```
T(n) = 2T(n - 1) + 1
T(n) = 2^2T(n - 2) + 2 + 1
T(n) = 2^3T(n - 3) + 2^2 + 2 + 1
```

Pattern:

```
T(n) = 2^kT(n - k) + 2^(k - 1) + ... + 2 + 1
```

Base case:

```
n - k = 0
k = n
```

Final:

```
T(n) = 2^n + 2^(n - 1) + ... + 2 + 1
```

GP sum:

```
T(n) = 2^(n + 1) - 1
```

Big O:

```
T(n) = O(2^n)
```

Stack space:

```
O(n)
```

The most important lesson is:

> Multiple decreasing recursive calls can make the running time grow exponentially.

---

32. What You Should Be Able to Do After Phase 3

After this phase, you should be able to:

1. Recognize a recursive function with multiple calls.
2. Identify the base case.
3. Count the number of recursive calls.
4. Track how the input changes.
5. Identify the extra work outside recursion.
6. Write the recurrence `T(n) = 2T(n - 1) + 1`.
7. Draw or understand the recursion tree.
8. Understand why the number of calls doubles at each level.
9. Solve the recurrence using successive substitution.
10. Use the base case equation `n - k = 0`.
11. Understand the GP sum `1 + 2 + 4 + ... + 2^n`.
12. Conclude that the time complexity is `O(2^n)`.
13. Understand why stack space is `O(n)`, not `O(2^n)`.
14. Avoid common mistakes such as counting only one recursive call.

---

33. Key Formula From Phase 3

Main recurrence:

```
T(n) = 2T(n - 1) + 1
T(0) = 1
```

After substitution:

```
T(n) = 2^kT(n - k) + 2^(k - 1) + ... + 2 + 1
```

Base case:

```
n - k = 0
k = n
```

Final expansion:

```
T(n) = 2^n + 2^(n - 1) + ... + 2 + 1
```

GP sum:

```
1 + 2 + 4 + ... + 2^n = 2^(n + 1) - 1
```

Final result:

```
Time: O(2^n)
Space: O(n)
```

This is the foundation for Phase 4, where this manually solved pattern becomes a shortcut using the Master Theorem for decreasing functions.